

Università degli Studi di Salerno
Corso di Ingegneria del Software

FumettoVerse
Object Design Document
Versione 1.1



Data: 21/12/2025

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Armando Pio La Rocca	0512117724
Davide Petracca	0512120550

Scritto da:	Armando Pio La Rocca
--------------------	----------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.0	Strutturazione documento Object Design Document	La Rocca e Petracca
21/12/2025	1.1	Revisione finale	La Rocca e Petracca

Sommario

1. INTRODUZIONE.....	3
1. Object Design Trade-offs.....	3
2. Linee guida per la documentazione delle interfacce.....	4
3. Definizioni, acronimi e abbreviazioni.....	4
4. Riferimenti.....	5
Design pattern.....	5
5. Singleton Pattern.....	5
6. MVC e DAO/Repository.....	5
Packages.....	5
7. Package core.....	7
L'implementazione di FumettoVerse è organizzata in due package principali Java e un insieme di pagine JSP lato view. 7	
8. Package model.....	7
9. Package control.....	8
10. JSP in WEB-INF.....	9
Class Interfaces.....	9
11. UserDAO.....	9
12. ComicDAO.....	10
13. OrderDAO.....	11
14. DatabaseConnection.....	12
15. Servlet principali in control.....	12
Glossario.....	16

1. INTRODUZIONE

Dopo la realizzazione dei documenti RAD e SDD di FumettoVerse sono stati definiti gli obiettivi del sistema, i requisiti funzionali e non funzionali, l'architettura logica three-tier/MVC e la decomposizione in sottosistemi.

Questo documento di Object Design ha lo scopo di produrre un modello che integri in modo coerente e preciso le funzionalità individuate, descrivendo i trade-off progettuali, le linee guida di codifica e le interfacce delle classi che realizzano la logica applicativa, il controllo e l'accesso ai dati persistenti

1. *Object Design Trade-offs*

- Affidabilità vs Flessibilità: il sistema privilegia l'affidabilità e la coerenza dei dati rispetto alla flessibilità nell'inserimento di dati parziali, in linea con i requisiti di robustezza e consistenza di checkout e gestione ordini.
- Accessibilità vs Controllo accessi: il catalogo è consultabile anche da utenti non registrati per massimizzare accessibilità e usabilità, mentre le operazioni sensibili (checkout, gestione catalogo, gestione ordini/utenti) sono riservate ad utenti autenticati con ruoli specifici.
- Spazio vs Velocità: sono privilegiate le prestazioni (es. indicizzazione delle colonne di ricerca, query ottimizzate) rispetto al risparmio di spazio, per garantire tempi di risposta contenuti per catalogo, carrello e ordini.

- Manutenibilità vs Rapidità iniziale: è stata adottata un'architettura modulare MVC/three-tier che separa Presentation, Application e Storage layer, aumentando l'effort iniziale ma semplificando estensioni e manutenzione nel tempo.

2. *Linee guida per la documentazione delle interfacce*

Naming convention

- Usare nomi descrittivi, pronunciabili, di uso comune, lunghezza medio-corta e non ambigui.
- Evitare abbreviazioni non standard; usare solo caratteri consentiti (a-z, A-Z, 0-9, underscore dove necessario).

Variabili

- I nomi iniziano con lettera minuscola, le parole successive con maiuscola (camelCase), es: `totalPrice`, `orderDate`.
- Dichiarare le variabili all'inizio del blocco, una per riga, allineate e se necessario accompagnate da un breve commento.

Metodi

- I nomi iniziano con minuscola e sono composti da verbo + oggetto, es: `addComic`, `findUserByEmail`.
- I metodi di accesso seguono la forma `getNomeVariabile()` / `setNomeVariabile()`.
- Ogni metodo è preceduto da un commento che ne descrive scopo, parametri, valore di ritorno ed eventuali eccezioni (es. `JavaDoc`).

Classi e pagine

- I nomi delle classi iniziano con lettera maiuscola, ogni parola interna in maiuscolo (es: `UserModel`, `OrderBean`, `CheckoutServlet`).
- La dichiarazione di classe segue l'ordine: dichiarazione, costanti, variabili di classe, variabili d'istanza, costruttori, metodi.

3. Definizioni, acronimi e abbreviazioni

- RAD: Requirements Analysis Document.
- SDD: System Design Document.
- ODD: Object Design Document.
- DBMS: Database Management System (MySQL).

- MVC: Model View Controller.

4. Riferimenti

- B. Bruegge, A. H. Dutoit, Object-Oriented Software Engineering – Using UML, Patterns, and Java.
- Documento RAD del progetto FumettoVerse.
- Documento SDD del progetto FumettoVerse.

Design pattern

Nel sistema FumettoVerse vengono adottati pattern coerenti con l'architettura descritta nel SDD.

5. Singleton Pattern

Il Singleton viene utilizzato per la gestione centralizzata delle connessioni verso il DBMS MySQL, tramite una classe di connection pool analoga a `DriverManagerConnectionPool` dell'esempio UnisaCinema.

La classe ha costruttore privato, un attributo statico che mantiene l'unica istanza e un metodo `getInstance()` statico che la restituisce, inizializzandola alla prima invocazione (lazy initialization)

6. MVC e DAO/Repository

- L'architettura MVC separa View (JSP/HTML/JS), Controller (Servlet) e Model (classi Java che incapsulano logica e accesso ai dati).
- Le classi Model/DAO (es. `UserModel`, `ComicModel`, `OrderModel`) espongono operazioni CRUD tipizzate e nascondono i dettagli JDBC/MySQL al resto del sistema.
- L'architettura MVC separa View (JSP/HTML/JS), Controller (Servlet) e Model (classi Java che incapsulano logica e accesso ai dati).
- Le classi Model/DAO (es. `UserModel`, `ComicModel`, `OrderModel`) espongono operazioni CRUD tipizzate e nascondono i dettagli JDBC/MySQL al resto del sistema.

Packages

Il nostro sistema presenta una suddivisione basata su tre livelli (three-tier):

- Interface layer
- Application Logic layer
- Storage layer.

Il package FumettoVerse contiene sottopackage che a loro volta inglobano classi atte alla gestione delle richieste utente. Le classi contenute nel package svolgono il ruolo di gestore logico del sistema.

Interface layer	Rappresenta l'interfaccia del sistema, ed offre la possibilità all'utente di interagire con quest'ultimo,
-----------------	---

	offrendo sia la possibilità di inviare, in input, che di visualizzare, in output, dati
--	--

Application Logic layer	Ha il compito di elaborare i dati da inviare al client, e spesso grazie a delle richieste fatte al database, tramite lo Storage Layer, accede ai dati persistenti. Si occupa di varie gestioni quali: ● Gestione Account ● Gestione Fumetti ● Gestione Manga ● Gestione Acquisti ● Gestione Carrello
-------------------------	---

Storage layer	Ha il compito di memorizzare i dati sensibili del sistema, utilizzando un DBMS, inoltre riceve le varie richieste dall' Application Logic layer inoltrandole al DBMS e restituendo i dati richiesti.
---------------	---

7. *Package core*

L'implementazione di FumettoVerse è organizzata in due package principali Java e un insieme di pagine JSP lato view.

- `control`: contiene le Servlet che realizzano la logica di controllo dell'applicazione e gestiscono il flusso tra client e modello.
- `model`: contiene le classi del modello (entità, DAO e gestione connessione al DB).
- JSP in `WEB-INF`: realizzano la parte di view e corrispondono alle principali funzionalità descritte in RAD/SDD (catalogo, carrello, checkout, account, ordini).

8. *Package model*

Contiene le classi che mappano le entità del dominio e le classi DAO che interagiscono con il database MySQL.

Classe	Ruolo
DatabaseConnection	Gestisce la connessione al DB (pattern Singleton / utility centralizzata).
Comic	Entità fumetto/manga: id, title, price, image, tipo, ecc.
ComicDAO	DAO per i fumetti: operazioni CRUD su tabella Comics.
User	Entità utente: id, name, email, password, isAdmin.
UserDAO	DAO per gli utenti: registrazione, login, ricerca, update/eliminazione.
Order	Entità ordine: id, user, orderNumber, orderDate, totalPrice, stato, ecc.
OrderItem	Entità riga d'ordine: riferimento al fumetto, quantità, prezzo a ordine.
OrderDAO	DAO per gli ordini: inserimento ordine, ricerca per utente, aggiornamento stato.

9. Package control

Le classi Servlet del package control implementano i casi d'uso (esplorazione catalogo, gestione carrello e ordini, gestione catalogo lato admin, autenticazione e registrazione).

Servlet	Descrizione
IndexServlet	Gestisce la home iniziale e reindirizza verso le pagine principali.
HomepageServlet (se presente)	Eventuale instradamento alla homepage.jsp.
AllComicsServlet	Recupera e mostra l'intero catalogo fumetti/manga (es. pagina allComics.jsp).
ComicsServlet	Gestisce visualizzazione dettagli di un singolo fumetto (pagina comics.jsp).
MangasServlet	Gestisce la vista dedicata alla categoria manga (manga.jsp).
CartServlet	Mostra il carrello (cart.jsp).
UpdateCartServlet	Aggiorna le quantità degli articoli nel carrello.
RemoveFromCartServlet	Rimuove un articolo dal carrello.
ClearCartServlet	Svuota completamente il carrello.
CheckoutServlet	Gestisce il processo di checkout e crea l'ordine (checkout.jsp → thankyou.jsp).
OrdersServlet	Mostra al cliente lo storico ordini (myorders.jsp).
LoginServlet	Gestisce autenticazione utente (login.jsp).
LogoutServlet	Termina la sessione utente.
RegisterServlet	Gestisce registrazione nuovo account (register.jsp).
ImageServlet	Restituisce immagini dei fumetti (es. thumbnail) come risposta HTTP.
GestoreCatalogServlet	Pannello admin per gestione catalogo (lista fumetti lato admin).
GestoreAddComicServlet	Inserisce un nuovo fumetto nel catalogo.
GestoreEditComicServlet	Modifica un fumetto esistente.

GestoreDeleteComicServlet	Elimina un fumetto dal catalogo.
GestoreOrdersServlet	Gestisce la visualizzazione/modifica stato ordini lato gestore.

10. JSP in WEB-INF

Le JSP implementano la view per i principali casi d'uso descritti nel RAD (catalogo, carrello, autenticazione, ordini).

- `index.jsp`: pagina di ingresso al sito, reindirizzamento o presentazione iniziale.
- `homepage.jsp`: home con categorie, novità e link a catalogo.
- `allComics.jsp`: visualizzazione di tutto il catalogo fumetti.
- `comics.jsp`: pagina dettaglio singolo fumetto.
- `manga.jsp`: catalogo filtrato sui manga.
- `cart.jsp`: visualizzazione carrello, con quantità e subtotali.
- `checkout.jsp`: form di checkout e riepilogo ordine.
- `thankyou.jsp`: conferma ordine avvenuto con successo.
- `login.jsp`, `register.jsp`: gestione autenticazione e registrazione.
- `myorders.jsp`: storici ordini del cliente autenticato.
- `header.jsp`, `footer.jsp`: componenti riusabili di layout.

Class Interfaces

11. UserDAO

UserDAO::doSave(user)

Pre:

- `user != null && user.email != null && user.password != null && user.name != null`
- `users->forAll(u | u.email <> user.email)`

Post:

- `users->include(user)`

UserDAO::doRetrieveByEmail(email)

Pre:

- `email <> null`

Post:

- `result = users->select(u | u.email = email)`

UserDAO::checkLogin(email, password)

Pre:

- `email <> null && password <> null`

Post:

- `result = users->any(u | u.email = email and u.passwordValid(password))` or
`result = null`

12. ComicDAO

ComicDAO::doSave(comic)

Pre:

- `comic.id <> null && comic.title <> null && comic.price <> null && comic.image <> null && comic.tipo <> null`
- `not comics->exists(c | c.id = comic.id)`

Post:

- `comics->exists(c | c.id = comic.id)`

ComicDAO::doDelete(id)

Pre:

- `id <> null && comics->exists(c | c.id = id)`

Post:

- `not comics->exists(c | c.id = id)`

ComicDAO::doRetrieveByKey(id)

Pre:

- `id <> null && comics->exists(c | c.id = id)`

Post:

- `result = comics->select(c | c.id = id)`

ComicDAO::doRetrieveAll()

Pre:

- `true`

Post:

- `result = comics` (lista completa catalogo)

13. *OrderDAO*

OrderDAO::doSave(order, items)

Pre:

- `order != null && items->size() > 0`
- `order.user != null && order.totalPrice = items->collect(i | i.priceAtPurchase * i.quantity)->sum()`

Post:

- `orders->include(order)` and `orderItems->includesAll(items)`

OrderDAO::doRetrieveByUser(user)

Pre:

- `user != null`

Post:

- `result = orders->select(o | o.user = user)`

OrderDAO::updateStatus(orderId, newStatus)

Pre:

- orders->exists(o | o.id = orderId)

Post:

- orders->select(o | o.id = orderId).status = newStatus

14. *DatabaseConnection*

DatabaseConnection::getConnection()

Pre:

- singletonInstance != null

Post:

- result <> null and result.isOpen = true

15. *Servlet principali in control*

LoginServlet

LoginServlet::doPost(email, password)

Pre:

- email <> null && password <> null

Post:

- if UserDao.checkLogin(email,password) <> null then session.user = user
else errorMessage <> null

RegisterServlet

RegisterServlet::doPost(userData)

Pre:

- `userData.email <> null && userData.password <> null && userData.name <> null`
- `not users->exists(u | u.email = userData.email)`

Post:

- `users->include(newUser) and session.user = newUser`

AllComicsServlet

AllComicsServlet::doGet()

Pre:

- `true`

Post:

- `request.comicsList = ComicDAO.doRetrieveAll() and forward("allComics.jsp")`

CartServlet

CartServlet::doGet(user)

Pre:

- `user.authenticated = true`

Post:

- `view = "cart.jsp" and request.cartItems = session.cartItems`

UpdateCartServlet

UpdateCartServlet::doPost(user, itemsQuantities)

Pre:

- `user.authenticated = true && itemsQuantities->forAll(q | q >= 0)`

Post:

- `cartItems->forAll(ci | ci.quantity = itemsQuantities[ci.comic])`

RemoveFromCartServlet

RemoveFromCartServlet::doPost(user, comicId)

Pre:

- `user.authenticated = true && cartItems->exists(ci | ci.comic.id = comicId)`

Post:

- `not cartItems->exists(ci | ci.comic.id = comicId)`

CheckoutServlet

CheckoutServlet::doPost(user, paymentData)

Pre:

- `user.authenticated = true`
- `cartItems->exists(ci | ci.user = user)`

Post:

- `OrderDAO.doSave(order, itemsFromCart(user))`
- `cartItems->forAll(ci | ci.user <> user)`
- `forward("thankyou.jsp")`

OrdersServlet

OrdersServlet::doGet(user)

Pre:

- `user.authenticated = true`

Post:

- `request.orders = OrderDAO.doRetrieveByUser(user)` and
`forward("myorders.jsp")`

GestoreAddComicServlet

`AdminAddComicServlet::doPost(admin, comicData)`

Pre:

- `admin.isAdmin = true`
- `comicData.title <> null && comicData.price <> null && comicData.tipo <> null`

Post:

- `ComicDAO.doSave(newComic)`

GestoreEditComicServlet

`AdminEditComicServlet::doPost(admin, comicData)`

Pre:

- `admin.isAdmin = true && comics->exists(c | c.id = comicData.id)`

Post:

- `selectedComic.attributes = comicData`

GestoreDeleteComicServlet

`AdminDeleteComicServlet::doPost(admin, id)`

Pre:

- `admin.isAdmin = true && comics->exists(c | c.id = id)`

Post:

- `not comics->exists(c | c.id = id)`

GestoreOrdersServlet

AdminOrdersServlet::doPost(admin, orderId, newStatus)

Pre:

- `admin.isAdmin = true && orders->exists(o | o.id = orderId)`

Post:

- `orders->select(o | o.id = orderId).status = newStatus`

Glossario

- Checkout: processo di conferma dell'ordine e pagamento online.
- Fumetto/Manga: prodotto editoriale venduto tramite la piattaforma FumettoVerse.
- Catalogo: insieme dei fumetti/manga disponibili alla consultazione e all'acquisto.
- Carrello: contenitore virtuale che memorizza i prodotti selezionati dall'utente prima del checkout.
- Ordine: insieme di righe prodotto acquistate in un'unica transazione, con numero identificativo e stato.
- Cliente/Utente registrato: utente autenticato che può aggiungere prodotti al carrello, effettuare acquisti e visualizzare lo storico ordini.
- Gestore: utente amministrativo che gestisce catalogo, ordini e utenti.
- DBMS MySQL: sistema di gestione del database relazionale usato per persistere utenti, fumetti, ordini, carrelli.
- Servlet: componente Java lato server che elabora le richieste HTTP e produce risposte dinamiche.